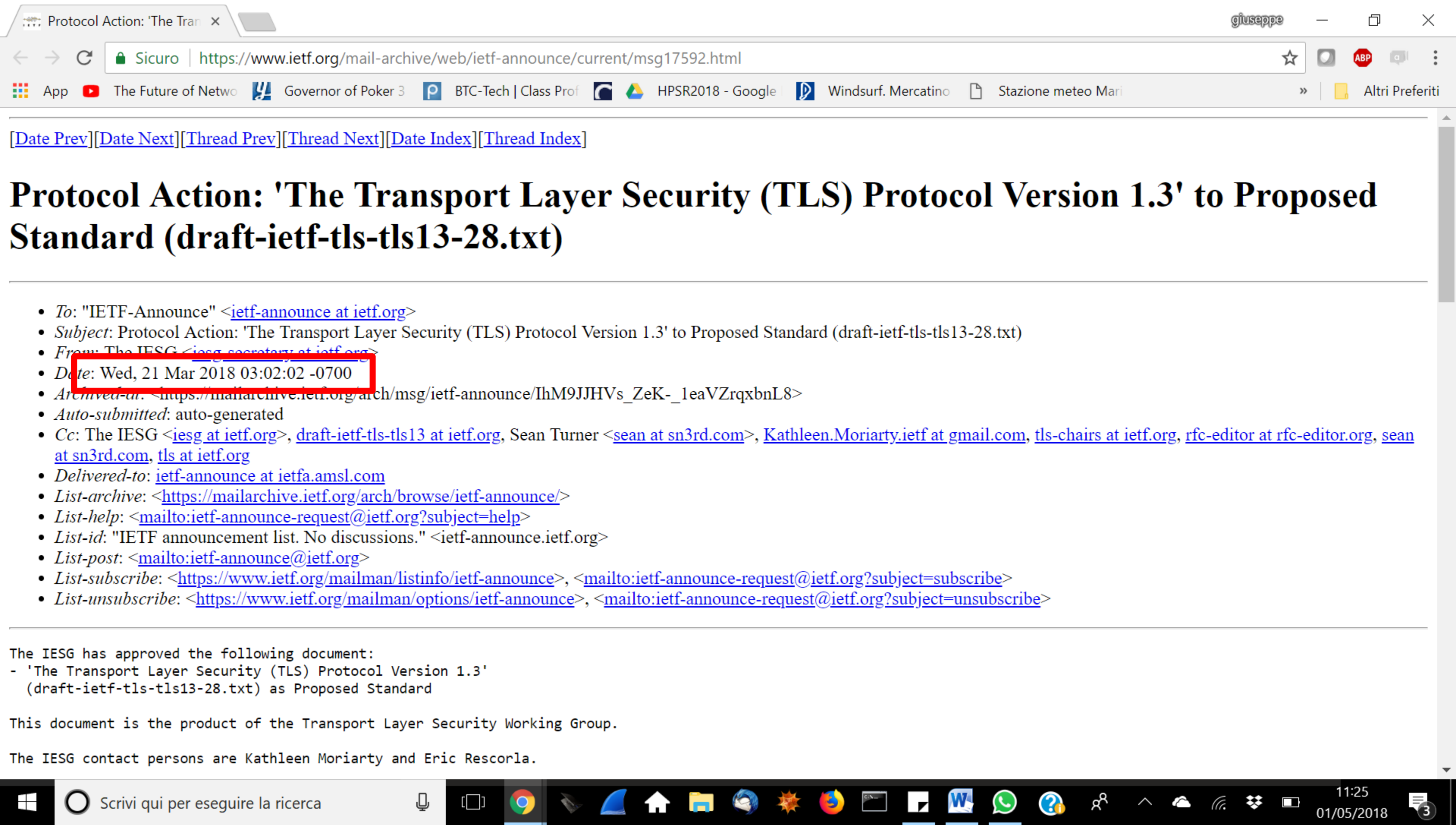
TLS 1.3 at a glance
&
Perfect Forward Secrecy

TLS 1.3: Approved in 2018



[Date Prev][Date Next][Thread Prev][Thread Next][Date Index][Thread Index]

Protocol Action: 'The Transport Layer Security (TLS) Protocol Version 1.3' to Proposed Standard (draft-ietf-tls-tls13-28.txt)

- *To*: "IETF-Announce" <ietf-announce at ietf.org>
- *Subject*: Protocol Action: 'The Transport Layer Security (TLS) Protocol Version 1.3' to Proposed Standard (draft-ietf-tls-tls13-28.txt)
- *From*: The IESG <iesg-secretary at ietf.org>
- *Date*: Wed, 21 Mar 2018 03:02:02 -0700
- *Archived-at*: <https://mailarchive.ietf.org/arch/msg/ietf-announce/IhM9JJHVs_ZeK-_1eaVZrqxbnL8>
- *Auto-submitted*: auto-generated
- *Cc*: The IESG <iesg at ietf.org>, draft-ietf-tls-tls13 at ietf.org, Sean Turner <sean at sn3rd.com>, Kathleen.Moriarty.ietf at gmail.com, tls-chairs at ietf.org, rfc-editor at rfc-editor.org, sean at sn3rd.com, tls at ietf.org
- *Delivered-to*: ietf-announce at ietfa.amsl.com
- *List-archive*: <https://mailarchive.ietf.org/arch/browse/ietf-announce/>
- *List-help*: <mailto:ietf-announce-request@ietf.org?subject=help>
- *List-id*: "IETF announcement list. No discussions." <ietf-announce.ietf.org>
- *List-post*: <mailto:ietf-announce@ietf.org>
- *List-subscribe*: <https://www.ietf.org/mailman/listinfo/ietf-announce>, <mailto:ietf-announce-request@ietf.org?subject=subscribe>
- *List-unsubscribe*: <https://www.ietf.org/mailman/options/ietf-announce>, <mailto:ietf-announce-request@ietf.org?subject=unsubscribe>

```
The IESG has approved the following document:
- 'The Transport Layer Security (TLS) Protocol Version 1.3'
  (draft-ietf-tls-tls13-28.txt) as Proposed Standard

This document is the product of the Transport Layer Security Working Group.

The IESG contact persons are Kathleen Moriarty and Eric Rescorla.
```

TLS 1.3: Approved in 2018

[Docs] [txt|pdf] [draft-ietf-tls-...] [Tracker] [Diff1] [Diff2] [IPR] [Errata]

PROPOSED STANDARD
Errata Exist

Internet Engineering Task Force (IETF)
Request for Comments: 8446
Obsoletes: 5077, 5246, 6961
Updates: 5705, 6066
Category: Standards Track
ISSN: 2070-1721

E. Rescorla
Mozilla
August 2018

The Transport Layer Security (TLS) Protocol Version 1.3

Abstract

This document specifies version 1.3 of the Transport Layer Security (TLS) protocol. TLS allows client/server applications to communicate over the Internet in a way that is designed to prevent eavesdropping, tampering, and message forgery.

This document updates RFCs 5705 and 6066, and obsoletes RFCs 5077, 5246, and 6961. This document also specifies new requirements for TLS 1.2 implementations.

Status of This Memo

This is an Internet Standards Track document.

This document is a product of the Internet Engineering Task Force (IETF). It represents the consensus of the IETF community. It has received public review and has been approved for publication by the Internet Engineering Steering Group (IESG). Further information on Internet Standards is available in Section 2 of RFC 7841.

TLS 1.3: weak ciphers' pruning

- **Pruned all symmetric algorithms considered legacy.**
  - SHA-1, MD5
  - RC4, DES, 3DES
  - AES-CBC
  - (plus a few more technical removals)
- **What is left?**
  - ALL (!) remaining ciphers are AEAD (Authenticated Encryption with Associated Data)
    - AEAD: no more CCA (Padding Oracle, Lucky 13. etc).
  - Notation: TLS_AEAD_HASH (hash used for HKDF):
    - TLS_AES_128_GCM_SHA256
    - TLS_AES_256_GCM_SHA384
    - TLS_CHACHA20_POLY1305_SHA256
    - TLS_AES_128_CCM_SHA256
    - TLS_AES_128_CCM_8_SHA256

TLS 1.3 handshake

- **TLS1.2: four pubkey handshake methods supported**
  - **RSA** key transport (most common)
  - **Anonymous** Diffie-Hellman
  - **Fixed** Diffie-Hellman
  - Diffie-Hellman **Ephemeral**

- **TLS1.3: removed all, except one!**
  - Diffie-Hellman **Ephemeral**

- **Why? Forward Secrecy!**
  - What's this? See next slides ☺
  - But also get rid of RSA issues (Bleichenbacker Oracle, …)

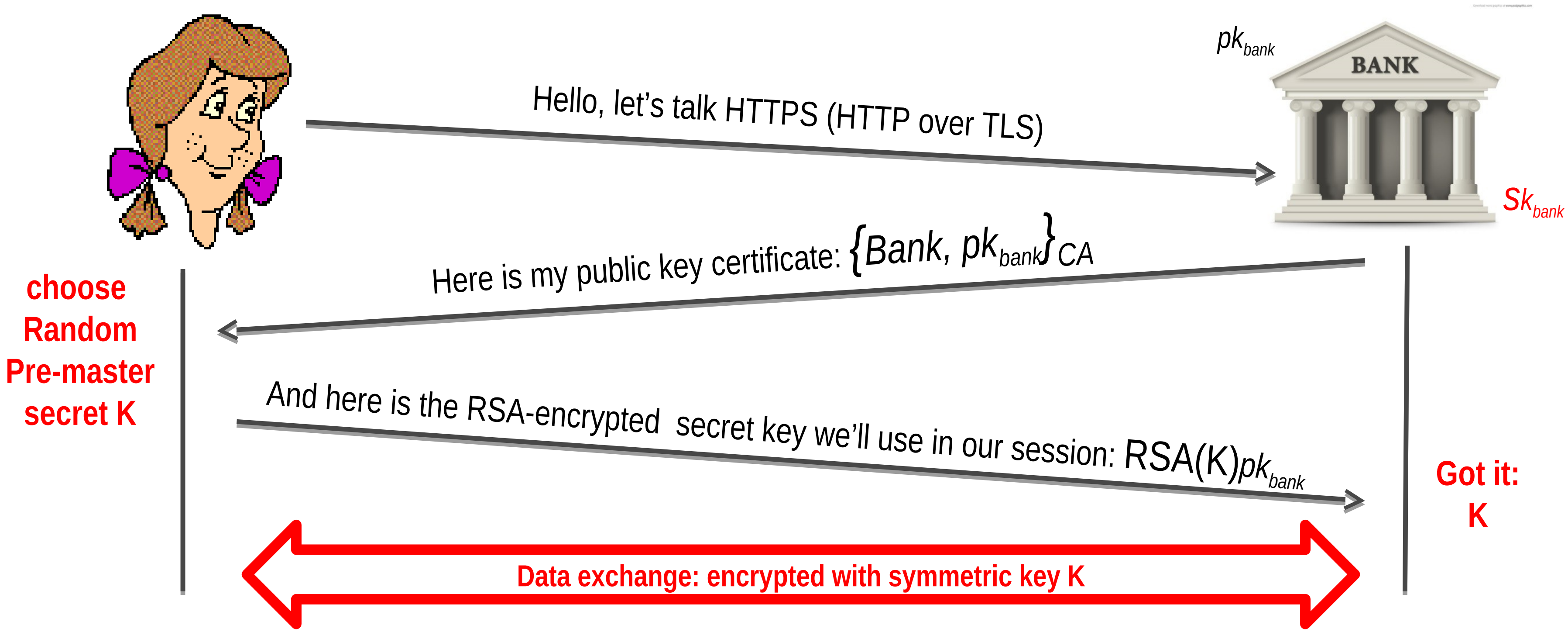
What is perfect forward secrecy?

- **At the very minimum:**
  - If a session key is compromised this should NOT compromise data exchanged neither BEFORE nor AFTER.
- **Actually more than this!**
  - if a long-term private key (e.g. of the server!!) is compromised at some time, this should also NOT affect data delivered BEFORE such time!
- **In essence: PFS guarantees that session keys will not be compromised even if long-term secrets used in the session key exchange are compromised**

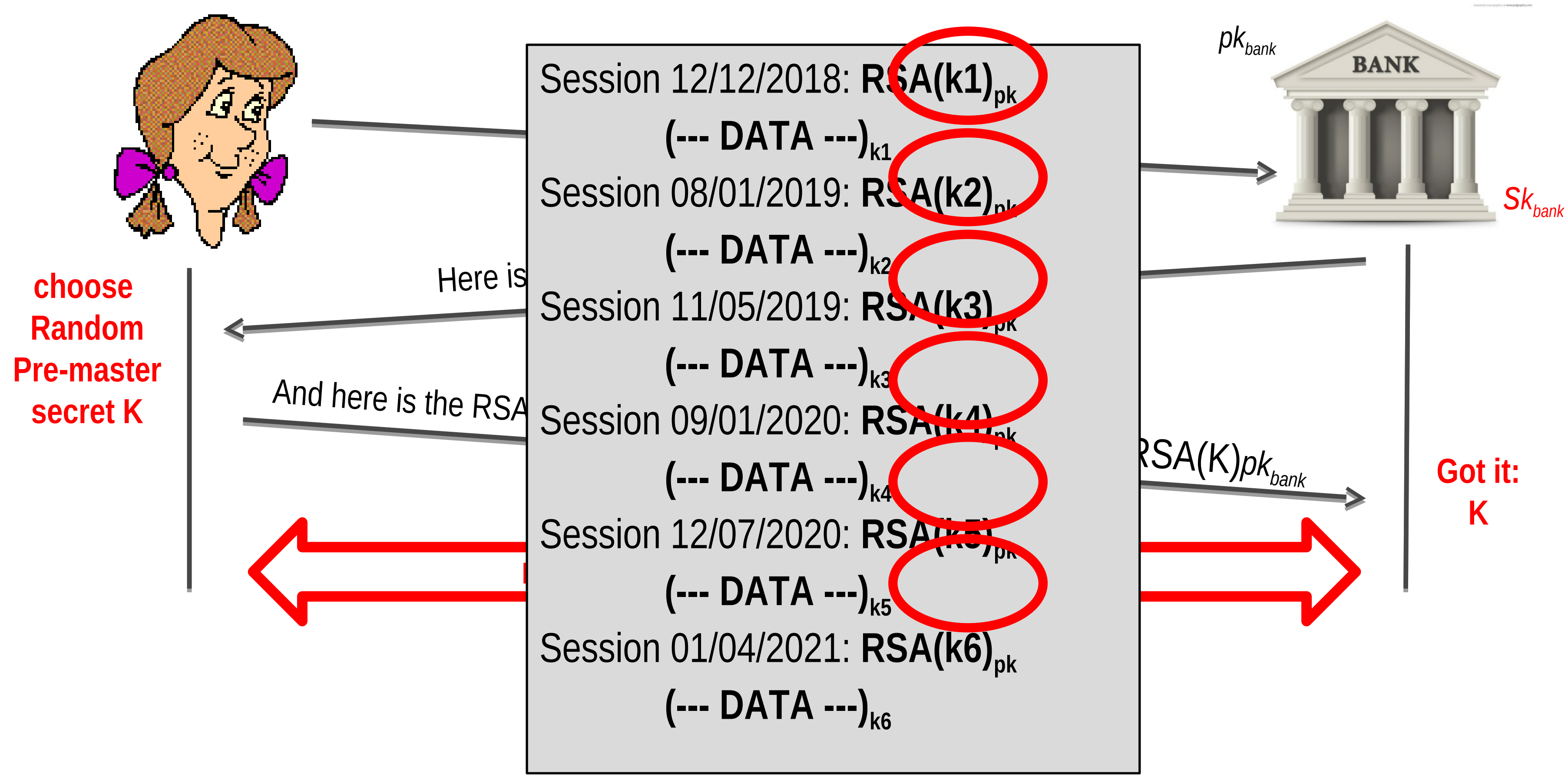
Why PFS is today more and more important?

- **Protects agains a more powerful attacker model**
  - Capable of **logging huge amount of data**
    - E.g. a mass-surveillance program capturing all network traffic (remember Snowden…)
  - Capable to **occasionally break a public/private key pair**
    - E.g. via a SW bug in the implementation – (remember heartbleed)

Traditional RSA key transport: no PFS!!

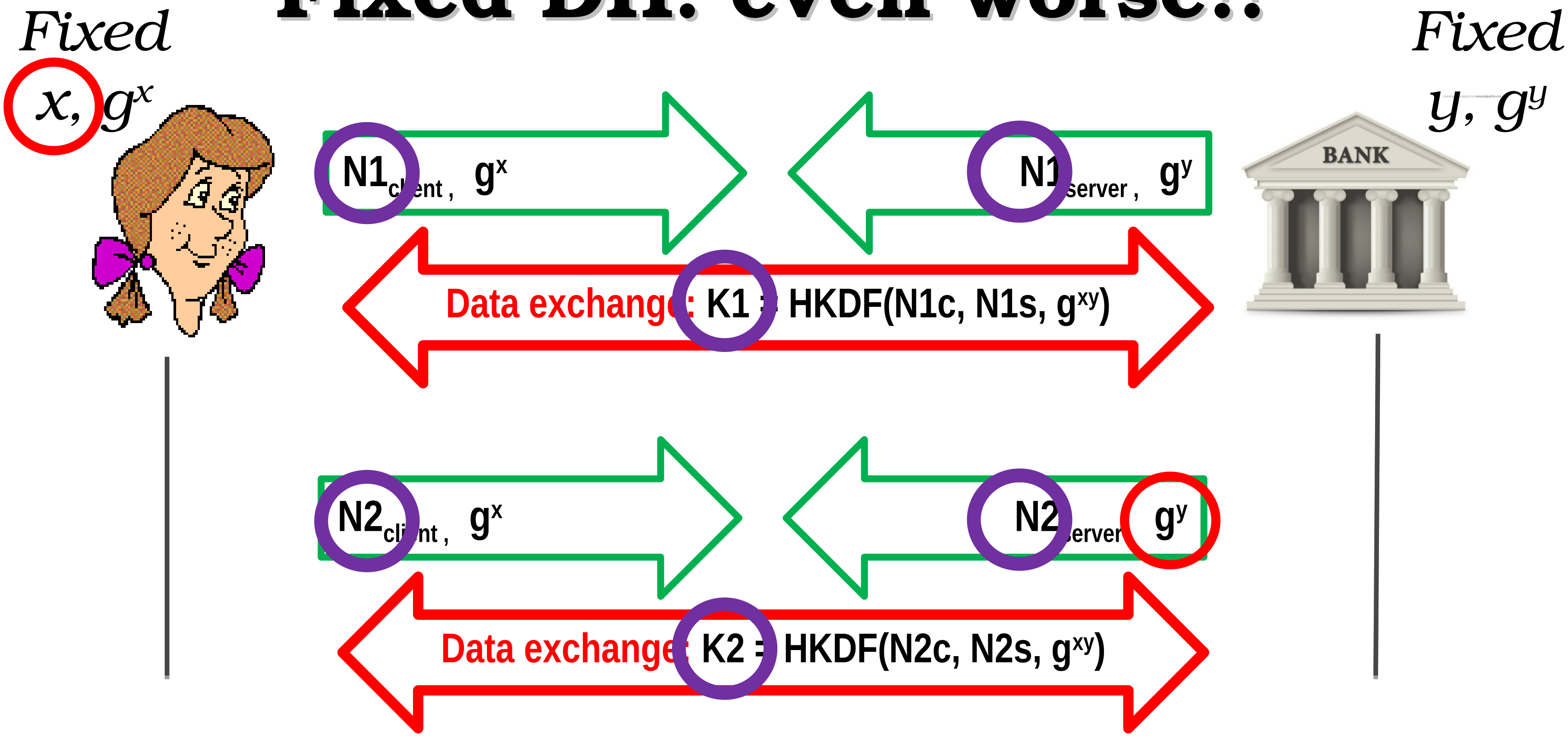


Traditional RSA key transport: no PFS!!



Breach 25/11/2021 → sk exposed

Fixed DH: even worse!!



Breach 25/11/2021→ x (or y) exposed → g^{xy} disclosed!

→ compute K2 (nonces in hello msg) → And all past K!!

Ephemeral Diffie-Hellman: PFS!

Alice

SK_A

choose random
$x \in \{0,1\}^s$

$(PK_A)_{CA}$

Send Public Key (cert) to permit verification

Signature to prevent MITM attack;

$(PK_B)_{CA}$

Bob

SK_B

choose random
$y \in \{0,1\}^s$

$(PK_A)_{CA}$ | signed$(g^x)_A$ mod p →

← $(PK_B)_{CA}$ | signed$(g^y)_B$ mod p

K= (g^{yx}) mod p

Data exchange: encrypted with K

K= (g^{xy}) mod p

If SK breaks, (ephemeral) session keys are not revealed! Forward Secrecy!

Amd even AFTER breach, if attacker if only passive (but MITM now possible)

What about PFS vs pre-shared key?

WPA2 key
K = 31EF21456

WPA2 key
K = 31EF21456

Session 1 encrypted with K

Session 2 encrypted with K

We must use same static K… how to get PFS then???

Add an anonymous DH exchange for ephemeral key gen!

$g^x \bmod p$ →

← $g^y \bmod p$

Session 3 encrypted with KE = $HKDF_K(g^{xy})$

$g^a \bmod p$ →

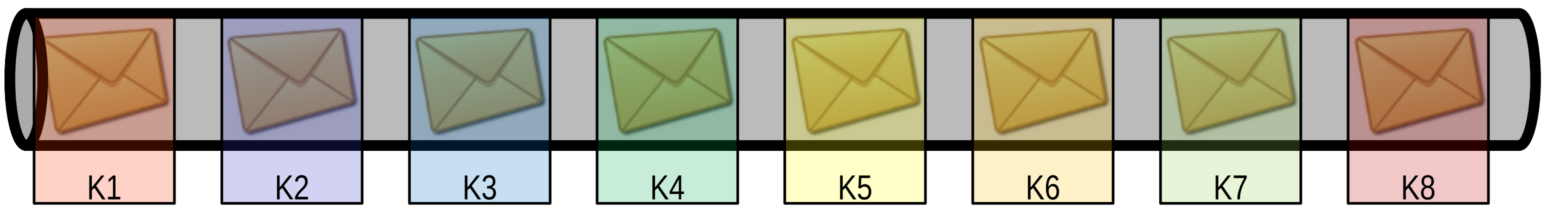
← $g^b \bmod p$

Session 4 encrypted with KE = $HKDF_K(g^{ab})$

… …

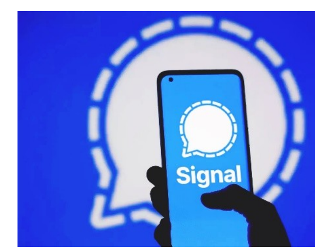
Forward Secrecy & messaging apps

~~E2e encryption: not even your provider can see~~

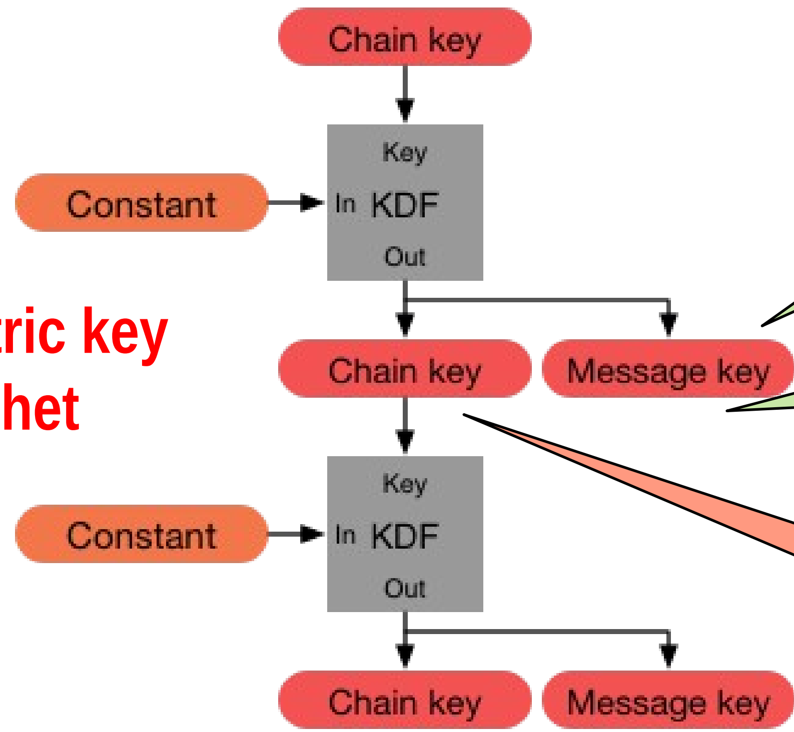


E2e encryption **with PFS**: Not only your provider cannot see,
But **even if it logs all data and gets access later on to your key, he cannot decrypt past messages!**

Signal's Double Ratchet: brilliant solution!



Double ratchet: combines a KDF chain…



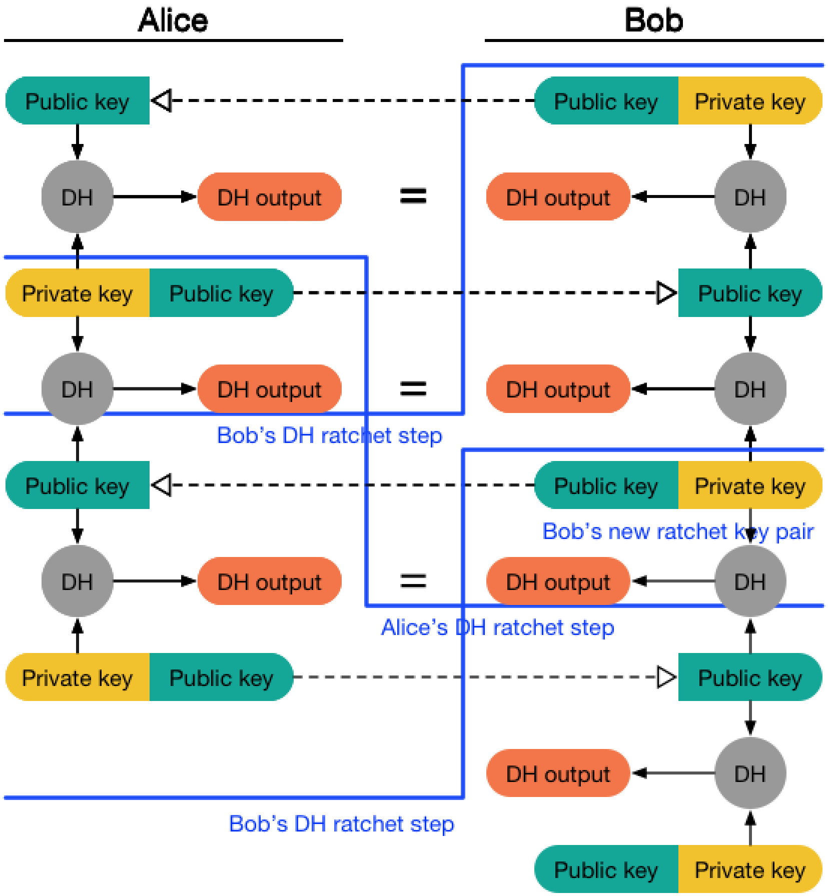
Symmetric key Ratchet

Encryption Key change @ every message!

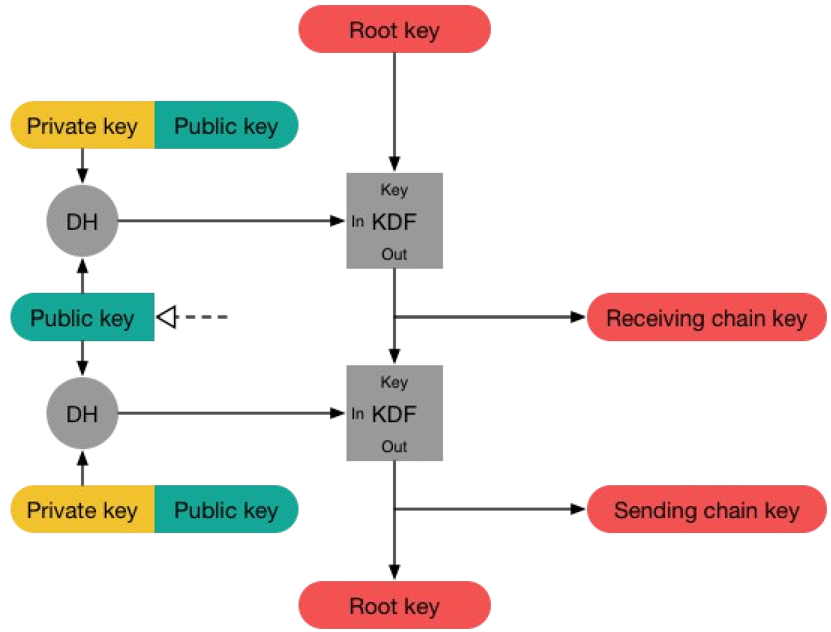
Discovering msg Encryption Key does not compromise (past or) future messages

But discovering chain key breaks future security…

…. With a DH ratchet, to guarantee forward secrecy!



Diffie-Hellman Ratchet



Double ratchet = Sym key ratchet + DH ratchet

TLS 1.3 handshake:
Faster and more secure!

- **With only DHE left (no more RSA transport):**
  - 3-way handshake versus 4-way! → faster setup
  - Perfect forward secrecy

→ **DHE public coefficient g^x, $Nonce_c$, ciphersuite offers (and groups)**

← **DHE public coefficient g^y, $Nonce_s$, selected mode**

← **Cert request, $cert_s$, signature with $cert_s$, Finished (HMAC)**

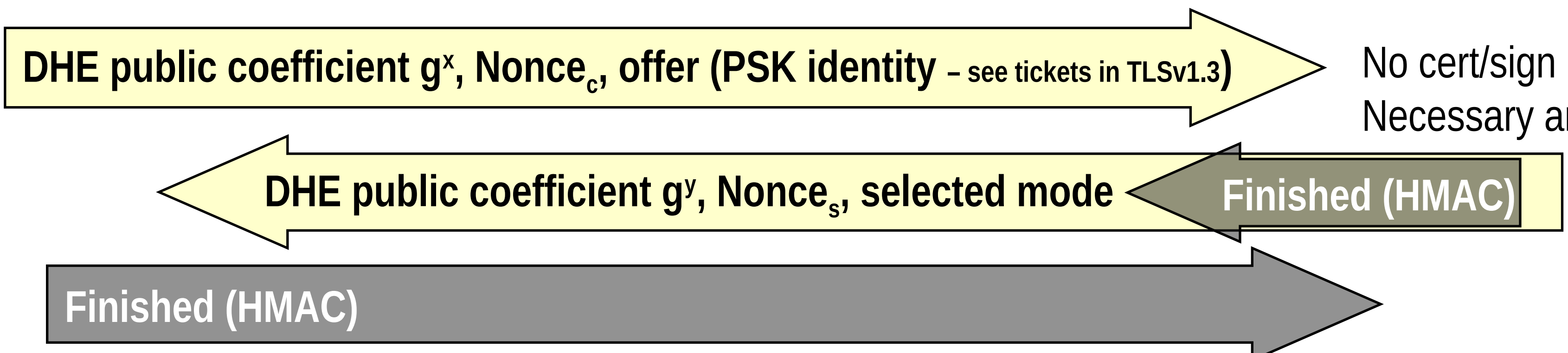
Already encrypted!

→ **$Cert_c$, signature with $cert_c$, Finished (HMAC)**

Side effect of early encryption: IDENTITY PROTECTION!
(certificates never transmitted in clear)
(not very useful in web, but keep in mind for different TLS scenarios…)

TLS 1.3 handshake:
Further options

➔ **Pre-Shared Key**
- ⇨ PSK, already added in TLS 1.2; PSK is either
  - → Agreed offline in constrained environments
  - → Computed during the first handshake and then reused in subsequent TSL connections
- ⇨ New in TLS1.3: optional support for forward secrecy
  - → Combines pre-shared key with a fresh DHE

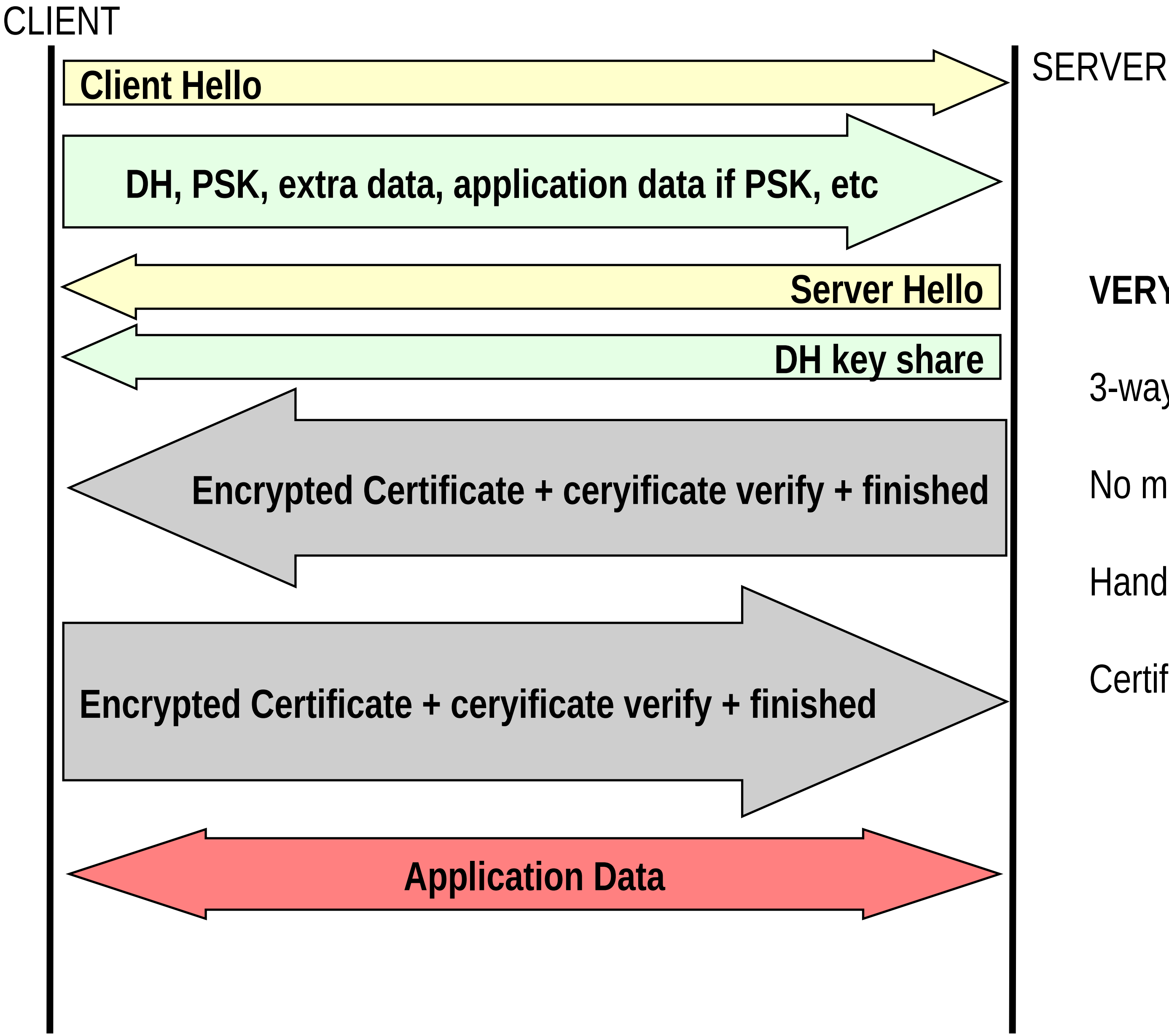


No cert/sign
Necessary anymore!

No cert/sign
Necessary anymore!

Session keys = HKDF(PSK, g^{xy}, etc) → inclusion of g^{xy} guarantees PFS

New handshake structure – wrap-up



VERY MUCH DIFFERENT!

3-way handshake

No more change Cipher specs

Handshake encrypted ASAP

Certificate + cert verify both sides

TLS 1.3 handshake:
Further options

- **0-RTT Data**
  - When PSK agreed (in previous handshake), send application data directly in first message
    - Use, as key: $HKDF_{PSK}$(Client-Hello-content)
    - i.e. «mix» PSK with new nonce, offer, and client's DHE public coefficient
  - MUCH faster!
    - Compelling use case: send HTTP request directly in very first message: saves a full RTT!
    - Also, very useful in applications relying on short data transfer
  - **WAY Less security (obvious replay attack): to be used with care… at your own risk!**

Mitigate replay in 0-RTT

- **Problem: key used in first msg does not include server nonce**
- **Time window mitigation**
  - Include maximum lifetime after first session
    - Standard default = 7 days
  - Does not solve… but at least reduces vulnerability period
- **Control nonce reuse**
  - Keep a record of all client nonces
  - Hard to deploy in large scale systems
    - Load balancing, multiple servers, etc…

Other TLS 1.3 nits
take home: less is more in security

- **No renegotiation anymore**
  - Key Upgrade feature, when rekeying ***inside*** same TLS session is required
- **HKDF officially included**
- **A few simplification in EC management**
  - Single EC point format, no negotiation
- **Removal of compression**
  - What else? Remember CRIME ☺
- **Exported key [integrates RFC 5705]**
  - Permits to export to applications a further shared secret key
    - Exporter master secret
    - Prevents application developers to “invent” their own approach!!
  - different and independent from the session master key!